

第 26 讲实验设计文档

NumPy 手写 Backpropagation + Training Loop

1 实验目的

本实验不是追求模型性能，而是验证一个最低深度学习科研闭环：

Forward $\rightarrow$ *Loss* $\rightarrow$ *Backward* $\rightarrow$ *Gradient Check* $\rightarrow$ *Update* $\rightarrow$ *Training Curve*

要求全部核心计算由 NumPy 手写完成，不调用 PyTorch Autograd。

2 研究问题与假设

RQ1: 前两讲推导出的两层 MLP Backward，在 Batch 情况下是否数值正确？

H1: 解析梯度与中央差分数值梯度应高度一致，误差接近浮点精度。

RQ2: 如果使用正确梯度并沿负梯度方向更新，Training MSE 是否持续下降？

H2: 在合适 learning rate 下，Loss 将从初始值显著下降，预测逐渐接近 $y = 2x + 1$ 。

RQ3: Learning rate 是否会改变优化稳定性和下降速度？

H3: 过小 learning rate 下降慢；适中 learning rate 更快；过大 learning rate 可能发散。

3 实验对象

数据

$X = 20$ 个等间隔点, $x \in [-1, 1]$

监督目标：

$$Y = 2X + 1$$

Shape:

$$X : (20, 1), \quad Y : (20, 1)$$

模型

两层 MLP:

$$1 \rightarrow 8 \rightarrow 1$$

$$Z_1 = XW_1 + b_1$$

$$A_1 = \text{ReLU}(Z_1)$$

$$\hat{Y} = A_1W_2 + b_2$$

Loss

$$MSE = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2$$

4 控制变量

Learning-rate 对照实验中固定：

- 数据完全相同；
- 网络结构完全相同；
- seed=42；
- 参数初始化完全相同；
- Loss 定义完全相同；
- Backward 实现完全相同。

只改变 learning rate，避免混杂因素。

5 实验 1：Gradient Check

选择四个代表性参数元素：

- $W_1[0, 0]$ ；
- $b_1[0, 0]$ ；
- $W_2[0, 0]$ ；
- $b_2[0, 0]$ 。

中央差分：

$$g_{num} \approx \frac{L(\theta + \epsilon) - L(\theta - \epsilon)}{2\epsilon}, \quad \epsilon = 10^{-5}$$

实际结果

参数	Analytic	Numeric	Abs Error
$W_1[0, 0]$	0.007174907	0.007174907	2.52×10^{-12}
$b_1[0, 0]$	0.013030021	0.013030021	1.46×10^{-11}
$W_2[0, 0]$	-0.445614491	-0.445614491	7.12×10^{-13}
$b_2[0, 0]$	-1.258798161	-1.258798161	5.55×10^{-12}

结果支持 H1：手写 Backward 与数值差分一致。

6 实验 2：完整 Training Loop

主实验设置：

- learning rate = 0.03；
- steps = 2000；

- `seed = 42`;
- full-batch training。

每一步：

1. Forward;
2. 计算 MSE;
3. Backward;
4. 记录 gradient norm;
5. $\theta \leftarrow \theta - \eta g$ 。

实际结果

Step	Loss	Gradient Norm
0	1.376293897	2.636107121
10	0.201688394	1.162390070
50	0.016090193	0.094922050
100	0.008488307	0.051503567
500	0.001005813	0.012892456
1000	0.000169043	0.003213296
1999	0.000065037	0.001765084

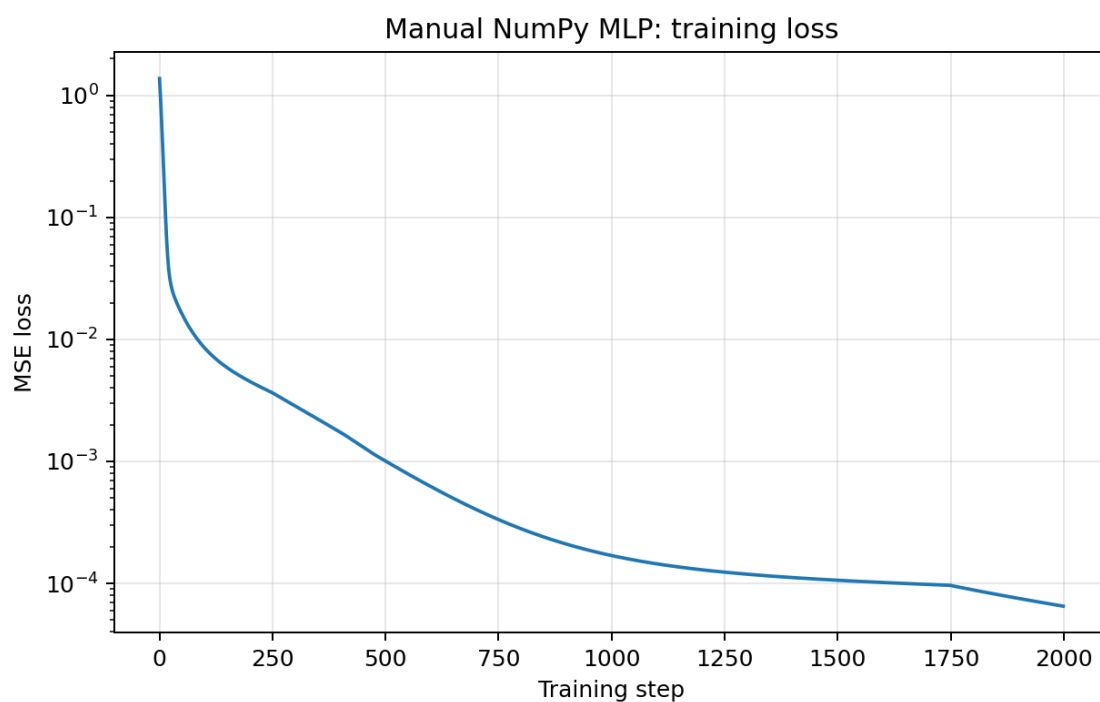


图 1: 主实验 Training Loss。

最终：

$$MAE = 0.004993288$$

$$MaxAbsError = 0.023183712$$

支持 H2。

7 实验 3：训练前后预测对比

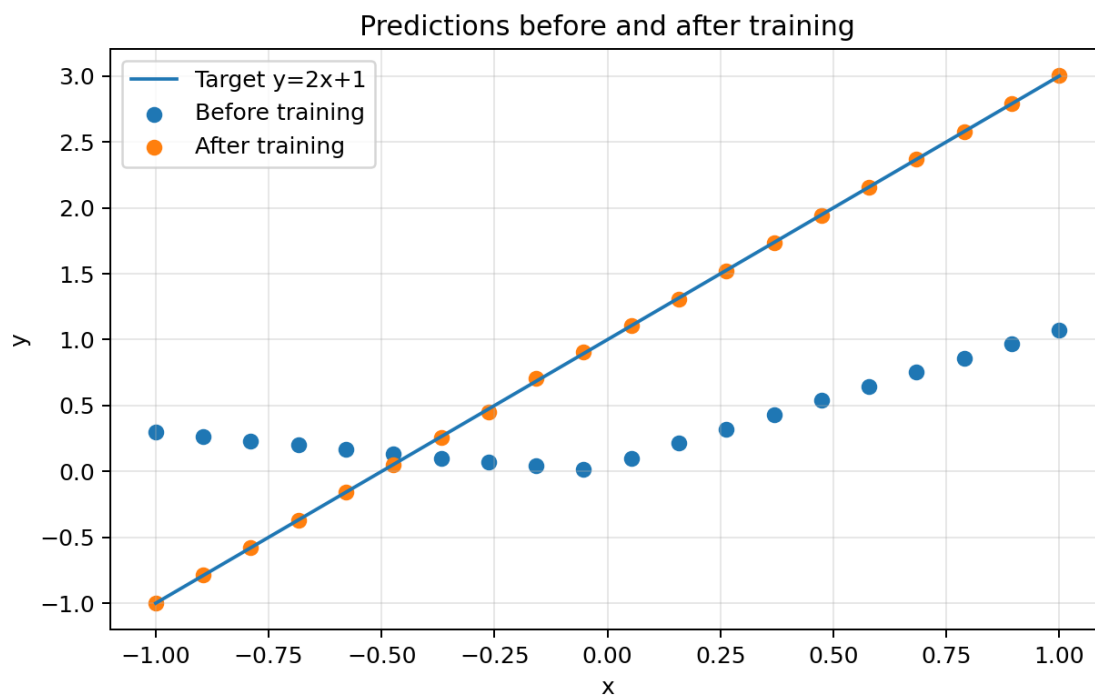


图 2: 训练前预测与训练后预测。

训练后预测已经非常接近目标直线。但该结果仅针对训练样本。

8 实验 4：Learning Rate 对照

同一初始化下：

LR	Steps	Initial Loss	Final Loss
0.003	500	1.376293897	0.016321767
0.03	500	1.376293897	0.001010820
1.0	80	1.376293897	10.717153391

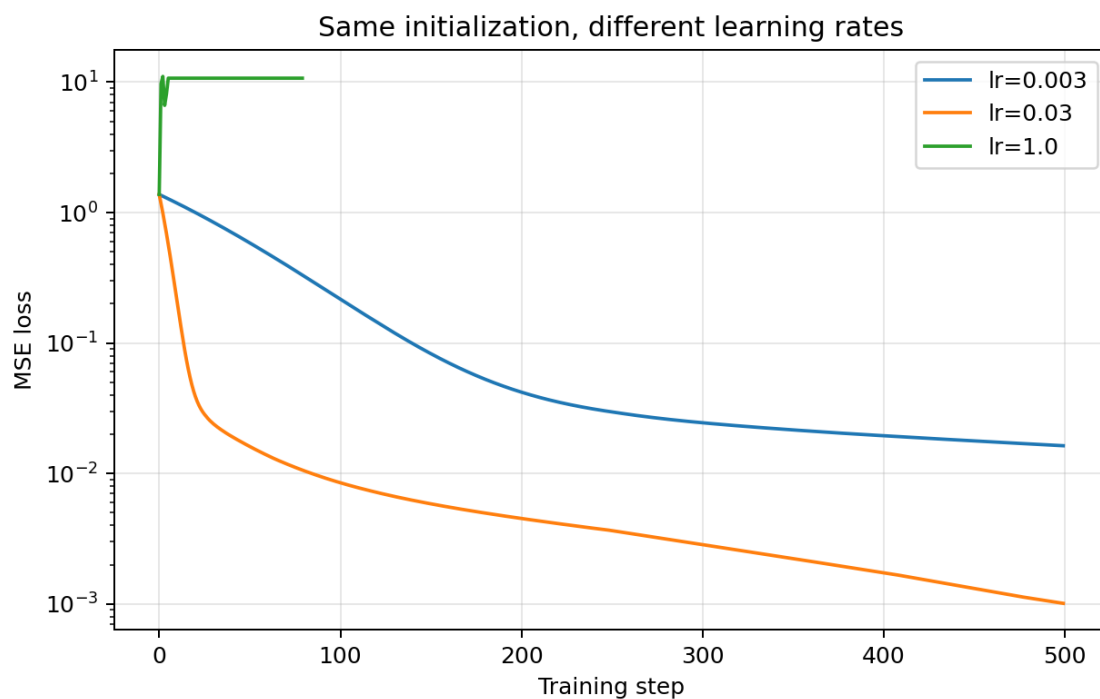


图 3: 固定其他条件, 只改变 learning rate。

结果支持 H3: 过大 learning rate 明显不稳定。

9 实验 5: Gradient Norm 记录

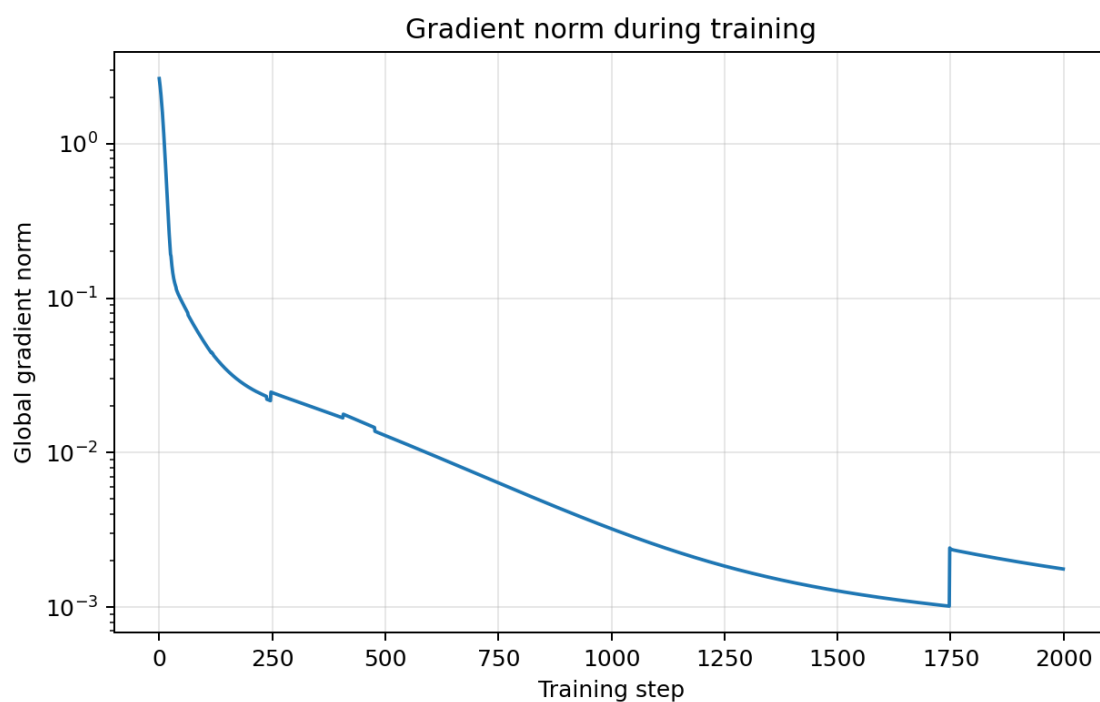


图 4: 主实验中的整体 Gradient Norm。

用途不是把 gradient norm 当成“好坏分数”，而是观察优化动态，辅助排查 exploding / vanishing / plateau 等现象。

10 关键实现检查清单

检查项	为什么重要
MSE 梯度是否除以 N	mean reduction 决定梯度尺度
$dW_2 = A_1^T d\hat{Y}$	转置错误会导致 Shape 或数值错误
db 是否沿 Batch 轴求和	Bias 被所有样本共享
ReLU backward 是否使用 Forward 的 Z_1	gate 来自 pre-activation 的正负
更新是否使用负号	Gradient 指向局部上升方向
Gradient check 是否通过	在训练前验证手写 backward
固定 seed	提高实验可复现性

11 结果解释边界

可以说：

- 当前手写 gradient 与 numerical gradient 一致；
- 当前训练设置下 MSE 能稳定下降；
- learning rate 对优化行为有显著影响。

不能说：

- 模型已经“泛化良好”；
- 0.03 是该类任务普遍最佳 learning rate；
- 两层 MLP 是最佳模型；
- 更低 Training Loss 一定意味着更好真实任务性能。

12 复现实验步骤

1. 确保安装 Python 3、NumPy、Matplotlib；
2. 打开终端进入材料包目录；
3. 运行：python lesson26_numpy_training_loop_experiment.py；
4. 对照 第 26 讲 _ 实验实际输出.txt；
5. 检查 figures 目录的四张图；
6. 尝试把 MAIN_LR 改成 0.003 或 1.0，观察变化；
7. 不要直接修改多处变量；一次只改变一个实验因素。

13 下一步

第 27 讲进入 PyTorch Tensor。届时将保留本讲心智模型，只把“自己管理数组与梯度”的部分逐步交给 PyTorch，从而明确框架到底自动化了哪些工作。